

CMake

CMake 개요

- CMake란?
플랫폼별 빌드 시스템(예: Makefile, Ninja, Visual Studio 솔루션)을 생성
CMakeLists.txt에 프로젝트 정보를 선언하고, CMake가 그에 맞는 빌드 구성
- Out-of-Source Build 권장
소스 폴더 바깥에 build 폴더를 만들고 그 안에서 CMake를 실행
나중에 정리나 재구성이 쉬워지고, 빌드 산출물(CMakeCache.txt, CMakeFiles 등)이
소스 트리에 섞이지 않음

CMake 개요

- 모던 CMake 지향

전역 설정 대신 타깃 기반 접근(target-based).

실행 파일/라이브러리를 타깃으로 정의하고, 그 타깃에만 필요한 include 경로, 링크, 컴파일 옵션을 부여 → 관리가 쉽고 의존성이 명확해짐

- 컴파일러/제네레이터 선택

여러 버전이 설치된 환경에선 CMake 실행 시 컴파일러를 명시할 수 있음.

예를 들어 CC, CXX 환경변수 또는 -D 인자를 사용해 C/C++ 컴파일러를 지정

CMakeLists.txt 주요 심볼(1)

심볼	의미
cmake_minimum_required	프로젝트를 빌드하기 위해 필요한 최소 CMake 버전을 지정 cmake_minimum_required(VERSION 3.16)
project	프로젝트 이름과 언어를 정의 project(MyApp LANGUAGES CXX)
add_executable	실행 파일을 생성 add_executable(main main.cpp utils.cpp)
add_library	라이브러리를 생성 (STATIC, SHARED, INTERFACE) STATIC 기본, 단 set(BUILD_SHARED_LIBS ON)시 SHARED 기본 add_library(mylib STATIC lib.cpp)
target_link_libraries	대상(target)에 다른 라이브러리 연결 target_link_libraries(main PRIVATE mylib)
include_directories	헤더 파일 검색 경로 추가 include_directories(\${CMAKE_SOURCE_DIR}/include)
find_package	외부 패키지/라이브러리 검색 find_package(OpenCV REQUIRED)
target_include_directories	특정 target에만 헤더 경로 추가 target_include_directories(main PRIVATE \${OpenCV_INCLUDE_DIRS})

CMakeLists.txt 주요 심볼(2)

심볼	의미
set	변수 정의 SET(변수명 값) or SET(변수명 값1 값2 값3) \$변수, 혹은 "{"를 써서 \${변수}로 이용 예1) SET(SRC_FILES a.c b.c) 예2) set(CMAKE_CXX_STANDARD 17)
option	ON/OFF 설정 옵션 정의 option(USE_MYLIB "Use my custom library" ON)
if / else / endif	조건문 cmake\nif(USE_MYLIB)\n add_subdirectory(mylib)\nendif()\n
message	빌드 시 메시지 출력 message(STATUS "Building project...")
add_subdirectory	하위 디렉토리 포함 add_subdirectory(src)
install	빌드 결과물 설치 규칙 정의 install(TARGETS main DESTINATION bin)

기본적인 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.22)
project>HelloCMake LANGUAGES CXX VERSION 0.1.0)
```

project : 이름, 언어, 버전까지 명시 가능

```
add_executable( hello main.cpp )
```

실제 프로그램은 add_executable/add_library로 만듦

```
# 권장: 표준/경고 등 컴파일 옵션은 타깃에 부여
```

```
target_compile_features(hello PRIVATE cxx_std_17)
```

빌드와 실행

소스 디렉토리로 이동

\$ cd tutorial

빌드 디렉토리 생성

\$ mkdir build && cd build

build 폴더만 지우면 깨끗한 재구성이 가능

CMake 실행 (상위 소스 지정)

\$ cmake ..

빌드 수행

\$ cmake --build .

실행 (Linux/macOS)

\$./hello

컴파일러 명시

1회성 지정 (첫 cmake 실행에만 유효)

```
CC=/usr/bin/gcc-11 CXX=/usr/bin/g++-11 cmake ..
```

캐시 변수로 지정 (유지됨)

```
cmake -D CMAKE_C_COMPILER=/usr/bin/gcc-11 -D CMAKE_CXX_COMPILER=/usr/bin/g++-11 ..
```

시스템이 원하는 컴파일러를 자동 선택하지 못할 때 명시

모던 CMake 핵심: 타깃 기반 접근

- 타깃 정의

`add_executable` 또는 `add_library`로 타깃을 만든 뒤, 그 타깃에 필요한 모든 속성을 국소적으로 부여

- include 경로

`target_include_directories`를 사용.

범위는 PRIVATE/INTERFACE/PUBLIC 중 선택

- PRIVATE: 현재 타깃을 빌드할 때만 필요.
- INTERFACE: 현재 타깃을 사용하는 다른 타깃에만 필요(헤더 전파 등).
- PUBLIC: 현재 타깃과 그 타깃을 사용하는 다른 타깃 모두에 필요.

- 링크

`target_link_libraries`로 연결하며, 상대 타깃 또는 시스템/패키지 라이브

모던 CMake 핵심: 타깃 기반 접근

- 라이브러리 + 실행 파일 예시

```
# 라이브러리 정의
add_library(mathFunctions
    src/mathFunctions.cpp
    include/mathFunctions.hpp
)
# 라이브러리의 헤더 경로 전파: 사용처가 포함해야 하므로 INTERFACE/ PUBLIC 사용
target_include_directories(mathFunctions
    PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)
# 실행 파일 정의
add_executable(tutorial main.cpp)
# 실행 파일에 라이브러리 연결
target_link_libraries(tutorial PRIVATE mathFunctions)
```

왜 모던 방식인가: 전역 include_directories, add_definitions 등은 프로젝트 전체에 퍼져 관리가 어렵기 때문에 타깃에만 필요한 정보를 부여하면 의존성 경로와 옵션이 자연스럽게 전파되고, 문제 추적이 용이

외부 라이브러리, 테스트, 패키징

- 외부 라이브러리 찾기와 연결

find_package: 설치된 패키지를 탐색하고 타깃을 가져옴

```
find_package(fmt CONFIG REQUIRED) # 예: fmt::fmt 타깃 제공
```

```
target_link_libraries(tutorial PRIVATE fmt::fmt)
```

가져온 타깃을 그대로 링크하면 include 경로/컴파일 옵션/정의 등이 함께 전파됨

CMake

- 단위 테스트 추가
 - 추천: CTest + GoogleTest 조합

```
enable_testing()

add_executable(math_tests tests/math_tests.cpp)
target_link_libraries(math_tests PRIVATE mathFunctions GTest::gtest_main)

add_test(NAME math_tests COMMAND math_tests)
```

- 실행: `ctest -V`로 테스트, CI와도 쉽게 연동

CMake

- 설치와 배포(기초)

```
# 설치 규칙
install(TARGETS tutorial mathFunctions
        RUNTIME DESTINATION bin
        LIBRARY DESTINATION lib
        ARCHIVE DESTINATION lib)

install(DIRECTORY include/ DESTINATION include)
```

cmake --install 로 시스템에 배포

예) `cmake --install build --prefix /my/install/path`

실무 팁과 트러블슈팅

- 빌드 폴더만 삭제

구성 초기화가 필요할 때 build 폴더를 통째로 지우면 깨끗한 상태로 재 시작 가능

- 경로에 공백

Windows에서 공백이 있는 경로를 변수로 전달할 땐 큰따옴표 사용

- 컴파일러 강제 지정

시스템이 Clang/GCC를 뒤섞어 선택하거나 구식 버전을 잡을 때는 CC/CXX 또는 CMAKE_*_COMPILER로 명시해 혼선을 최소화

- 전역 설정 지양

include_directories, link_directories, add_definitions로 모든 타깃에 영향을 주지 말 것
대신 target_* API로 필요한 범위에만 적용

실무 팁과 트러블슈팅

- 프로젝트 선언 습관

루트 CMakeLists.txt에서 project(...)를 명시적으로 선언하면 IDE/제네레이터가 올바른 솔루션/메타 정보를 구성

- Out-of-Source 습관화

실수로 소스 트리에 빌드 산출물을 포함시키지 말 것. 이 원칙 하나만 지켜도 유지보수가 쉬워짐

실습 1: CMake로 Hello World 실행 파일 만들기

- 디렉토리 구조

```
HelloProject/  
|—— CMakeLists.txt  
|—— src/  
|    |—— main.cpp  
|—— build/ ← 빌드 전용 폴더
```

- main.cpp 작성 (src/main.cpp)

```
#include <iostream>  
  
int main() {  
    std::cout << "Hello, World!" << std::endl;  
    return 0;  
}
```


실습1: CMake로 Hello World 실행 파일 만들기

- CMakeLists.txt 작성

```
cmake_minimum_required(VERSION 3.10)
project>HelloProject)

set(CMAKE_CXX_STANDARD 17)

add_executable(hello src/main.cpp)
```

- 빌드 및 실행 과정

```
$ cd helloProject
$ mkdir build
$ cd build
$ cmake ..
$ cmake --build .
$ ./hello
```

실습2: 라이브러리 분리(1)

- 디렉토리 구조

```
tutorialProject/  
|—— CMakeLists.txt  
|—— tutorial.cpp  
|—— mathFunctions/  
|   |—— CMakeLists.txt  
|   |—— mathFunctions.h  
|   |—— mathFunctions.cpp  
|—— build/
```

실습2: 라이브러리 분리(2)

- mathFunctions 라이브러리

```
#pragma once

namespace mathFunctions {
    double square(double x);
}
```

mathFunctions/mathFunctions.h

```
#include "mathFunctions.h"

namespace mathFunctions {
    double square(double x) {
        return x * x;
    }
}
```

mathFunctions/mathFunctions.cpp

```
add_library(mathFunctions mathFunctions.cpp)
```

mathFunctions/CMakeLists.txt

```
# 헤더 경로 전파
target_include_directories(mathFunctions
    PUBLIC ${CMAKE_CURRENT_SOURCE_DIR} # 라이브러리를 사용하는 대상에게 헤더 경로 제공
)
```

PUBLIC: 라이브러리를 사용하는 대상에게 헤더 경로를 전파
INTERFACE: 라이브러리 자체에는 필요 없지만, 사용하는 대상에게만 경로 제공.
PRIVATE: 라이브러리 내부에서만 사용.

실습2: 라이브러리 분리(3)

- 소스 파일

```
#include <iostream>
#include "mathFunctions.h"

int main() {
    std::cout << "Square of 5 is " << mathFunctions::square(5) << std::endl;
    return 0;
}
```

tutorial.cpp

```
cmake_minimum_required(VERSION 3.10)
project(tutorialProject)

set(CMAKE_CXX_STANDARD 17)

add_executable(tutorial tutorial.cpp)

# mathFunctions 서브디렉토리 추가
add_subdirectory(mathFunctions)

# tutorial 실행 파일에 라이브러리 링크
target_link_libraries(tutorial PRIVATE mathFunctions)
```

CMakeLists.txt

실습2: 라이브러리 분리(4)

- 빌드 및 실행

```
cd tutorialProject  
mkdir build  
cd build  
cmake ..  
cmake --build .  
./tutorial
```

실습3: 정적/공유 라이브러리 빌드 및 설치(1)

- 디렉토리 구조

```
tutorialProject/  
|—— CMakeLists.txt  
|—— tutorial.cpp  
|—— mathFunctions/  
|   |—— CMakeLists.txt  
|   |—— mathFunctions.h  
|   |—— mathFunctions.cpp  
|—— build/
```

실습3: 정적/공유 라이브러리 빌드 및 설치(2)

- mathFunction 라이브러리

```
# STATIC 또는 SHARED 선택 가능
option(BUILD_SHARED_LIBS "Build using shared libraries" ON)

add_library(mathFunctions mathFunctions.cpp)

target_include_directories(mathFunctions
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:include>
)

# 설치 규칙
install(TARGETS mathFunctions
    EXPORT mathFunctionsTargets
    ARCHIVE DESTINATION lib
    LIBRARY DESTINATION lib
    RUNTIME DESTINATION bin)

install(FILES mathFunctions.h DESTINATION include)
```

mathFunctions/CMakeLists.txt

실습3: 정적/공유 라이브러리 빌드 및 설치

- 루트 디렉토리

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(tutorialProject)

set(CMAKE_CXX_STANDARD 17)

add_executable(tutorial tutorial.cpp)

add_subdirectory(mathFunctions)

target_link_libraries(tutorial PRIVATE mathFunctions)

# tutorial 실행 파일 설치 규칙
install(TARGETS tutorial
        RUNTIME DESTINATION bin)

# CMake 패키지 설정 (라이브러리 export)
install(EXPORT mathFunctionsTargets
        FILE mathFunctionsTargets.cmake
        NAMESPACE mathFunctions::
        DESTINATION lib/cmake/mathFunctions)
```


실습3: 정적/공유 라이브러리 빌드 및 설치(3)

- 빌드 및 설치

```
$ cd tutorialProject
$ mkdir build
$ cd build
$ cmake .. -DBUILD_SHARED_LIBS=ON # 공유 라이브러리 빌드
$ cmake --build .
$ cmake --install . --prefix ../install
```

- 설치 후 구조

```
install/
├── bin/
│   └── tutorial
├── lib/
│   ├── libmathFunctions.a # STATIC 빌드 시
│   ├── libmathFunctions.so # SHARED 빌드 시
│   └── cmake/
│       └── mathFunctions/
│           └── mathFunctionsTargets.cmake
└── include/
    └── mathFunctions.h
```

관련사이트

- <https://cmake.org/cmake/help/latest/guide/tutorial/index.html#>
- [1. Overview — CGold 0.1 documentation](#)
- [CMake 할때 쪼오오금 도움이 되는 문서 · GitHub](#)
- <https://modoocode.com/332>